



## **Experiment No. 2**

**Title: Implementation of removal of punctuations, stop words, extra white spaces, URLs and HTML code from Text**



Batch: NLP-1

Experiment No.: 02

Roll No.: 16010422234

Name: Chandana Galgali

---

**Aim of the Experiment:** To implement removal of punctuations, stop words, extra white spaces, URLs and HTML code from Text.

---

**Program/Steps:**

1. Add custom list of stop words to English language stop words and use this list of stop words to remove stop words from text
2. Apply stop word removal, punctuation removal, space removal, URL and HTML code removal to a dataset of technical discussion forum such as dataset of stack overflow.

```
import os
os.environ['NLTK_DATA'] = r"C:/Users/Dell/nltk_data"

import re
import argparse
from pathlib import Path

import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

import pandas as pd

# -----
# 0) Safe NLTK bootstrap
# -----

def ensure_nltk():
    try:
        nltk.data.find("tokenizers/punkt")
    except LookupError:
        nltk.download("punkt", quiet=True)
    try:
        nltk.data.find("corpora/stopwords")
    except LookupError:
        nltk.download("stopwords", quiet=True)

ensure_nltk()
```

```

# -----
# 1) Helper: I/O paths
# -----

def make_outputs_dir():
    out_dir = Path("outputs")
    out_dir.mkdir(exist_ok=True)
    return out_dir

# -----
# 2) Cleaning primitives (per your procedure)
# -----

URL_PATTERN =
r"(http|ftp|https)://([\w_-]+(?:([\w_-]+)+))([\w.,@?^=%&:/~+#-]*[\w@?^
=%&/~+#-])?"

HTML_PATTERN = r"<[^>]+>"

def remove_extra_whitespace(text: str) -> str:
    text = text.strip()
    text = " ".join(text.split())
    return text

def remove_urls(text: str) -> str:
    return re.sub(URL_PATTERN, "", text)

def remove_html(text: str) -> str:
    return re.sub(HTML_PATTERN, "", text)

def tokenize(text: str):
    return word_tokenize(text)

def remove_punct(tokens):
    import string
    return [t for t in tokens if t not in string.punctuation]

def remove_stop_words(tokens, extra_custom=None):
    sw = set(stopwords.words("english"))
    # Add your custom stopwords here (demonstration list per "Activity")
    custom = {"stackoverflow", "amp", "im", "ive", "dont", "doesnt",
"didnt"}
    if extra_custom:
        custom.update(extra_custom)

```

```

sw.update(custom)
return [t for t in tokens if t.lower() not in sw]

def extract_urls(text: str):
    """
    For the 'Questions' part: identify and extract URLs from text data.
    Returns list of full URL strings detected BEFORE removal.
    """
    # Return full matched URLs (not groups)
    # Use finditer to capture the entire match
    matches = re.finditer(URL_PATTERN, text)
    return [m.group(0) for m in matches]

# -----
# 3) Audio → text (+ punctuation restoration)
# -----

def transcribe_audio_with_punctuation(audio_path: str):
    """
    For the 'Questions' part: sample code to extract text from audio and
    add punctuation.

    Uses SpeechRecognition for ASR, then deepmultilingualpunctuation to
    restore punctuation.

    If punctuation model isn't available, a simple rule-based fallback
    adds a final period.
    """
    import speech_recognition as sr

    recognizer = sr.Recognizer()
    with sr.AudioFile(audio_path) as source:
        audio_data = recognizer.record(source)

    # 1) Speech-to-text via Google's free web API (requires internet)
    try:
        raw_text = recognizer.recognize_google(audio_data)
    except Exception as e:
        raw_text = ""
        print(f"[Audio] Recognition failed: {e}")

    punctuated = None
    if raw_text:
        # 2) Try punctuation restoration model

```

```

        try:
            from deepmultilingualpunctuation import PunctuationModel
            model = PunctuationModel()
            punctuated = model.restore_punctuation(raw_text)
        except Exception as e:
            print(f"[Audio] Punctuation model not available. Fallback
used. ({e})")

            # Fallback: naive end punctuation if missing
            stripped = raw_text.strip()
            punctuated = stripped if stripped.endswith(('.', '!', '?'))
else stripped + '.'

        return raw_text, punctuated

# -----
# 4) Pipeline runner
# -----

def process_text_block(raw_text: str, show_snapshots=True):
    out_dir = make_outputs_dir()

    # Snapshot 0: Extract URLs (before removing them)
    urls_found = extract_urls(raw_text)
    if urls_found:
        with open(out_dir / "extracted_urls.txt", "w", encoding="utf-8")
as f:
            for u in urls_found:
                f.write(u + "\n")

    if show_snapshots:
        print("\n[SNAPSHOT] Example of extracted URLs (first 5):",
urls_found[:5])

    # Step A: Remove extra whitespace
    s1 = remove_extra_whitespace(raw_text)
    if show_snapshots:
        print("\n[SNAPSHOT] After whitespace removal (first 300 chars):")
        print(s1[:300])

    # Step B: Remove URLs
    s2 = remove_urls(s1)
    if show_snapshots:
        print("\n[SNAPSHOT] After URL removal (first 300 chars):")
        print(s2[:300])

```

```

# Step C: Remove HTML tags
s3 = remove_html(s2)
if show_snapshots:
    print("\n[SNAPSHOT] After HTML removal (first 300 chars):")
    print(s3[:300])

# Step D: Tokenize
toks = tokenize(s3)
if show_snapshots:
    print("\n[SNAPSHOT] Tokenized sample (first 30 tokens):")
    print(toks[:30])

# Step E: Remove punctuation
toks_no_punct = remove_punct(toks)
if show_snapshots:
    print("\n[SNAPSHOT] After punctuation removal (first 30 tokens):")
    print(toks_no_punct[:30])

# Step F: Remove stop words (with custom list added)
toks_no_sw = remove_stop_words(toks_no_punct)
if show_snapshots:
    print("\n[SNAPSHOT] After stopword removal (first 30 tokens):")
    print(toks_no_sw[:30])

# Join back (optional)
cleaned_text = " ".join(toks_no_sw)

# Save artifacts
with open(out_dir / "cleaned_text.txt", "w", encoding="utf-8") as f:
    f.write(cleaned_text)
with open(out_dir / "cleaned_tokens.txt", "w", encoding="utf-8") as f:
    for t in toks_no_sw:
        f.write(t + "\n")

return cleaned_text, toks_no_sw

def process_csv(csv_path: str, text_col: str):
    out_dir = make_outputs_dir()
    df = pd.read_csv(csv_path)
    if text_col not in df.columns:
        raise ValueError(f"Column '{text_col}' not found in CSV.
Available: {list(df.columns)}")

```

```

# Apply cleaning to each row and keep snapshots for first row
cleaned_list = []
token_list = []
first_row_snapshot_done = False

for idx, row in df.iterrows():
    raw = str(row[text_col]) if pd.notna(row[text_col]) else ""
    cleaned_text, toks = process_text_block(raw, show_snapshots=(not
first_row_snapshot_done))
    cleaned_list.append(cleaned_text)
    token_list.append(" ".join(toks))
    if not first_row_snapshot_done:
        first_row_snapshot_done = True

df["cleaned_text"] = cleaned_list
df["cleaned_tokens"] = token_list
df.to_csv(out_dir / "cleaned_dataset.csv", index=False,
encoding="utf-8")
print(f"\n[OK] Saved cleaned dataset -> {out_dir /
'cleaned_dataset.csv'}")

def process_txt(txt_path: str):
    with open(txt_path, "r", encoding="utf-8") as f:
        raw_text = f.read()
    process_text_block(raw_text, show_snapshots=True)
    print(f"\n[OK] Saved cleaned files into /outputs")

# -----
# 5) CLI
# -----

def main():
    parser = argparse.ArgumentParser(description="NLP Cleaning Experiment:
stopwords, punctuation, whitespace, URLs, HTML.")
    parser.add_argument("--csv", type=str, help="Path to CSV file (Stack
Overflow style).")
    parser.add_argument("--text-col", type=str, default="post_text",
help="Column name containing text (default: post_text).")
    parser.add_argument("--txt", type=str, help="Path to plain text
file.")
    parser.add_argument("--audio", type=str, help="Path to audio .wav file
for ASR + punctuation (optional).")

```

```

args = parser.parse_args()

# Enforce either CSV or TXT (one of them must exist)
if not args.csv and not args.txt and not args.audio:
    raise SystemExit("Provide --csv <path> (and --text-col) or --txt
<path> or --audio <file>. You can provide multiple.")

# Text cleaning from CSV or TXT
if args.csv:
    if not os.path.exists(args.csv):
        raise SystemExit(f"CSV file not found: {args.csv}")
    print(f"[INFO] Processing CSV: {args.csv} (column:
{args.text_col})")
    process_csv(args.csv, args.text_col)

if args.txt:
    if not os.path.exists(args.txt):
        raise SystemExit(f"Text file not found: {args.txt}")
    print(f"[INFO] Processing TXT: {args.txt}")
    process_txt(args.txt)

# Audio transcription (optional, for Q&A part)
if args.audio:
    if not os.path.exists(args.audio):
        raise SystemExit(f"Audio file not found: {args.audio}")
    print(f"[INFO] Transcribing audio: {args.audio}")
    raw, punct = transcribe_audio_with_punctuation(args.audio)
    out_dir = make_outputs_dir()
    with open(out_dir / "audio_transcript_raw.txt", "w",
encoding="utf-8") as f:
        f.write(raw or "")
    with open(out_dir / "audio_transcript_punctuated.txt", "w",
encoding="utf-8") as f:
        f.write(punct or "")
    print("\n[SNAPSHOT] Audio transcript (raw):", (raw or "")[:250])
    print("\n[SNAPSHOT] Audio transcript (punctuated):", (punct or
"")[:250])
    print(f"\n[OK] Saved audio transcripts to {out_dir}")

if __name__ == "__main__":
    main()

```

## Output/Result:

```

PS C:\Users\Dell\Downloads\VII SEM\HO-NLP\LAB\EXP-2> & C:/Users/Dell/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/Dell/Downloads/VII SEM/HO-NLP/LAB/EXP-2/EXP-2.py" --csv sample.csv --text-col post_text
[INFO] Processing CSV: sample.csv (column: post_text)

[SNAPSHOT] Example of extracted URLs (first 5): ['https://docs.oracle.com/javase/8/docs/api/']

[SNAPSHOT] After whitespace removal (first 300 chars):
<p>How do I fix 'NullPointerException' in Java? </p> I tried this code: <code>obj.toString()</code> but obj is None... Docs: https://docs.oracle.com/javase/8/docs/api/ Thanks!!!

<p>How do I fix 'NullPointerException' in Java? </p> I tried this code: <code>obj.toString()</code> but obj is None... Docs: Thanks!!!
<p>How do I fix 'NullPointerException' in Java? </p> I tried this code: <code>obj.toString()</code> but obj is None... Docs: Thanks!!!

[SNAPSHOT] After HTML removal (first 300 chars):
How do I fix 'NullPointerException' in Java? I tried this code: obj.toString() but obj is None... Docs: Thanks!!!

[SNAPSHOT] Tokenized sample (first 30 tokens):
['How', 'do', 'I', 'fix', 'NullPointerException', 'in', 'Java', 'I', 'tried', 'this', 'code', ':', 'obj.toString', '(', ')', 'but', 'obj', 'is', 'None', '...', 'Docs', ':', 'Thanks', '!', '!', '!']

[SNAPSHOT] After punctuation removal (first 30 tokens):
['How', 'do', 'I', 'fix', 'NullPointerException', 'in', 'Java', 'I', 'tried', 'this', 'code', 'obj.toString', 'but', 'obj', 'is', 'None', '...', 'Docs', 'Thanks']

[SNAPSHOT] After stopword removal (first 30 tokens):
['fix', 'NullPointerException', 'Java', 'tried', 'code', 'obj.toString', 'obj', 'None', '...', 'Docs', 'Thanks']

[OK] Saved cleaned dataset -> outputs\cleaned_dataset.csv
PS C:\Users\Dell\Downloads\VII SEM\HO-NLP\LAB\EXP-2>

```

## Post Lab Question-Answers:

As discussed there might be a need to add punctuations or retain URLs in the given Text.

1. Write the sample python code for extracting text from audio and add appropriate punctuations to it.

**Ans:** We can use the SpeechRecognition library to convert speech to text, and deepmultilingualpunctuation or similar models to restore punctuation.

```

import speech_recognition as sr
from deepmultilingualpunctuation import PunctuationModel

# Initialize recognizer
recognizer = sr.Recognizer()

# Load punctuation restoration model
model = PunctuationModel()

# Load audio file
with sr.AudioFile('audio_sample.wav') as source:
    audio_data = recognizer.record(source)

# Convert speech to text
text = recognizer.recognize_google(audio_data)

```

```
print("Without punctuation:", text)

# Restore punctuation
punctuated_text = model.restore_punctuation(text)
print("With punctuation:", punctuated_text)
```

The audio is transcribed using Google's Speech Recognition API. The transcribed text is passed through a punctuation restoration model to insert appropriate punctuation marks.

2. Write a sample python code to identify the URLs from the text data and extract URLs from text data.

**Ans:** We can use the re (regular expressions) module to find all URLs.

```
import re

# Sample text containing URLs
text = """
Natural language processing is a field of AI.
Check out https://en.wikipedia.org/wiki/NLP and http://example.com
for more info.
"""

# Regex pattern for URLs
url_pattern =
r"(http|ftp|https)://([\w_-]+(?:\:([\w_-]+)+))([\w.,@?^=%&:/~+#!-]*
[\w@?^=%&/~+#!-])?"

# Find all URLs
urls = re.findall(url_pattern, text)

# Extract full URLs from the regex groups
urls = ["".join(match) for match in urls]

print("Extracted URLs:", urls)
```

The regex pattern matches any string that starts with **http**, **https**, or **ftp** and follows standard URL formatting. `re.findall()` returns all matching URLs from the given text.

**Conclusion (based on the Results and Outcomes achieved):**

Through this experiment, we implemented and understood various text preprocessing techniques essential for NLP, including the removal of stop words, punctuations, extra white spaces, URLs, and HTML code. These steps helped in converting raw, unstructured text into a cleaner and more standardized form, making it suitable for further natural language processing tasks such as tokenization, sentiment analysis, and information retrieval. The activity demonstrated the practical use of Python libraries like NLTK and regular expressions for text cleaning, as well as the flexibility to customize preprocessing steps based on specific application needs. This not only improved the quality of the input text but also reinforced the importance of data cleaning in building efficient and accurate NLP models.

---

**References:**

**Books/ Journals/ Websites:**

1. Allen James, Natural Language Understanding, Benjamin Cumming, Second Edition, 1995
  2. Jurafsky, Dan and Martin, James, Speech and Language Processing, Prentice Hall, 2008
  3. Palash Goyal, Karan Jain, Sumit Pandey, Deep Learning for Natural Language Processing: Creating Neural Networks with Python, Apress, 2018
-